Homework 1

William Flanders*
Haverford College
(Dated: February 2, 2023)

1. NEWMAN EXERCISE 2.10

For Exercise 2.10 from Newman, we needed to form a python program that could output an element's bond energy that is described by

$$B = a_1 A - a_2 A^{\frac{2}{3}} - a_3 \frac{Z^2}{A^{\frac{1}{3}}} - a_4 \frac{(A-2Z)^2}{A} + \frac{a_5}{A^{\frac{1}{2}}} \quad (1)$$

Here, B represents bonding energy, A represents mass, Z represents atomic number, and a_1, a_2, a_3, a_4, and a_5 are constants. a_5 takes different values depending on the value of A and Z as given by

$$a_5 = \begin{cases} 0 & \text{if } A \text{ is odd,} \\ 12 & \text{if } A \text{ and } Z \text{ are both even,} \\ -12 & \text{if } A \text{ is even and } Z \text{ is odd} \end{cases}$$

Furthermore, we needed to look at elements 1 through 100, checking the largest bonding energy per nucleon described by

$$\frac{B}{A} \quad (2)$$

where we would use Equation 1 and plug in a range of values for Z ranging from the element's atomic number to three times its atomic number and outputting the largest bonding energy per nucleon and the corresponding mass.

We begin by outlining our logic into pseudo code. This helps with planning and implementation.

in the future, a diagram is better

```
for loop Z from 1 to 100:
    for loop A from Z to 3*Z:

        initialize constants and variables
        intialize largest B/A

        if A is even and Z is even:
            a_5 = 12
        if A is even and Z is odd:
            a_5 = -12
        else:
            a_5 = 0

        Calculate B

        Calculate B/A

        if current B/A > largest B/A:
            largest B/A = current B/A

    print("Largest B/A is " + str(B/A) +
          " for element " + str(Z))
```

After writing the code and executing, we now have the mass for each element from 1 to 100 that provides the largest bonding energy per nucleon. The results are summarized below

```
Z: 1 A: 3 B/A 0.36869091831015827
Z: 2 A: 4 B/A 5.321930578649441
Z: 3 A: 7 B/A 5.280168164356119
Z: 4 A: 8 B/A 6.466330085889912
Z: 5 A: 11 B/A 6.650123444727665
Z: 6 A: 14 B/A 7.200918138809924
Z: 7 A: 15 B/A 7.330860591990981
Z: 8 A: 18 B/A 7.719275577459026
Z: 9 A: 19 B/A 7.73697768275634
Z: 10 A: 22 B/A 8.035350864715019
Z: 11 A: 25 B/A 8.025554739665797
Z: 12 A: 26 B/A 8.241172535624845
Z: 13 A: 29 B/A 8.240988355754636
Z: 14 A: 30 B/A 8.37916169002579
Z: 15 A: 33 B/A 8.38521415855582
Z: 16 A: 36 B/A 8.489230168218935
Z: 17 A: 37 B/A 8.48201495174352
Z: 18 A: 40 B/A 8.573405285254953
Z: 19 A: 43 B/A 8.551826855569242
Z: 20 A: 44 B/A 8.627152167121634
Z: 21 A: 47 B/A 8.610130576802973
Z: 22 A: 48 B/A 8.6585154571142
Z: 23 A: 51 B/A 8.645234048730842
Z: 24 A: 54 B/A 8.687306583887372
Z: 25 A: 55 B/A 8.662703971015583
Z: 26 A: 58 B/A 8.701432576808987
Z: 27 A: 61 B/A 8.678053678353882
Z: 28 A: 62 B/A 8.70245768367189
Z: 29 A: 65 B/A 8.681907349580422
Z: 30 A: 68 B/A 8.701580328486784
Z: 31 A: 69 B/A 8.675012598311142
Z: 32 A: 72 B/A 8.693433639739787
Z: 33 A: 75 B/A 8.668156247337208
Z: 34 A: 76 B/A 8.67683411103597
Z: 35 A: 79 B/A 8.653727479263061
Z: 36 A: 82 B/A 8.66141248935323
```

*Electronic address: wflanders@haverford.edu

you don't need to print all of this in writeup

```
Z: 37 A: 85 B/A 8.633940444065898
Z: 38 A: 86 B/A 8.639441275530453
Z: 39 A: 89 B/A 8.613815033672552
Z: 40 A: 92 B/A 8.614514461544127
Z: 41 A: 93 B/A 8.587741675710747
Z: 42 A: 96 B/A 8.588337807352417
Z: 43 A: 99 B/A 8.561488033970447
Z: 44 A: 102 B/A 8.557428804696526
Z: 45 A: 103 B/A 8.531857077904819
Z: 46 A: 106 B/A 8.52785864674169
Z: 47 A: 109 B/A 8.500490244095234
Z: 48 A: 110 B/A 8.4940569666179
Z: 49 A: 113 B/A 8.46797678166812
Z: 50 A: 116 B/A 8.460713172533021
Z: 51 A: 119 B/A 8.433224407343122
Z: 52 A: 120 B/A 8.424696665334825
Z: 53 A: 123 B/A 8.398332486638505
Z: 54 A: 126 B/A 8.38868945344295
Z: 55 A: 129 B/A 8.361310553455423
Z: 56 A: 130 B/A 8.350819725669146
Z: 57 A: 133 B/A 8.32443069811602
Z: 58 A: 136 B/A 8.313019639868703
Z: 59 A: 139 B/A 8.285884693887583
Z: 60 A: 140 B/A 8.273583815729522
Z: 61 A: 143 B/A 8.247327458286065
Z: 62 A: 146 B/A 8.234582872513133
Z: 63 A: 149 B/A 8.207769087134613
Z: 64 A: 150 B/A 8.193813966434627
Z: 65 A: 153 B/A 8.167786207290451
Z: 66 A: 156 B/A 8.154024477520512
Z: 67 A: 159 B/A 8.127574530301825
Z: 68 A: 162 B/A 8.11278037767185
Z: 69 A: 163 B/A 8.086373082934724
Z: 70 A: 166 B/A 8.071829365612306
Z: 71 A: 169 B/A 8.045764583108769
Z: 72 A: 172 B/A 8.030368571593026
Z: 73 A: 175 B/A 8.004105311543137
Z: 74 A: 176 B/A 7.988369073415251
Z: 75 A: 179 B/A 7.962697427019249
Z: 76 A: 182 B/A 7.946838055684514
Z: 77 A: 185 B/A 7.921016888554693
Z: 78 A: 188 B/A 7.904575879934101
Z: 79 A: 191 B/A 7.87868400232803
Z: 80 A: 192 B/A 7.862438691993173
Z: 81 A: 195 B/A 7.837047201045294
Z: 82 A: 198 B/A 7.82033857608665
Z: 83 A: 201 B/A 7.794899333942829
Z: 84 A: 204 B/A 7.777785761025879
Z: 85 A: 205 B/A 7.75239433360284
Z: 86 A: 208 B/A 7.735485475322138
Z: 87 A: 211 B/A 7.710478810144077
Z: 88 A: 214 B/A 7.693222175964613
Z: 89 A: 217 B/A 7.668228396728228
Z: 90 A: 220 B/A 7.65068598823387
Z: 91 A: 223 B/A 7.625739835440575
Z: 92 A: 224 B/A 7.608214013689897
Z: 93 A: 227 B/A 7.583639834526779
Z: 94 A: 230 B/A 7.566035830526522
Z: 95 A: 233 B/A 7.5415108314640555
Z: 96 A: 236 B/A 7.523703637516345
Z: 97 A: 239 B/A 7.499251800171257
Z: 98 A: 242 B/A 7.481279349508352
Z: 99 A: 243 B/A 7.456937323389022
Z: 100 A: 246 B/A 7.439122944429214
```

We can see that as atomic number increases, so does bonding energy per nucleon until we hit element ~~26~~, and then bonding energy begins to fall again. 28, but code is correct

2. NEWMAN EXERCISE 2.2

We want to begin by deriving the height of a satellite that is orbiting the Earth. Since the force of gravity is equivalent to the centripetal force, we start with

$$F_g = F_c \tag{3}$$

We can plug in for these forces using Newton's Laws and we find that

$$\frac{GMm}{(h+R)^2} = m(\frac{2\pi}{T})^2(R+h) \tag{4}$$

where $G = 6.67 \cdot 10^{-11}\ m^3\ kg^{-1}\ s^{-2}$, $M = 5.97 \cdot 10^24$ kg and $R = 6371$ km Solving for h yields

10^{24}

$$h = (\frac{GMT^2}{4\pi^2})^{(1/3)} - R \tag{5}$$

Now that we have an equation in terms of height for the satellite, we can write a program that can find a satellite's height given its orbital period. We start by writing pseudo code to organize our program's logic

```
initialize constants
prompt user for period
store its value in a variable t

h = equation with respect to t
print("The satellite's height is " + str(h) +
" given its orbital period " + str(t))
```

Using the program, we find the height of a satellite with orbital periods of one day, 90 minutes, and 45 minutes respectively. The results are

```
One day     | 35855910.17617497
90 minutes  | 279321.62537285965
45 minutes  | -2181559.8978108233
```

Strangely, we can see that a period of 45 minutes yields a negative height for the satellite. This implies that in order for a satellite to have a period that is 45 minutes, it needs to orbit the Earth at a radius that is below ground.

For a true geosynchronous satellite, the orbital period is 23.93 hours. Plugging this into the program yeilds

```
23.93 hours | 35773762.329895645
24 hours    | 35855910.17617497
Difference  | 82147.84628
```

Because the Earth isn't a true sphere, it takes a little less than 24 hours for it complete a full rotation. This means that for an agent to release a satellite into space that perfectly coincides with the Earth's rotation, they would need it to be roughly 82 km closer to ground than if the period were a perfect 24 hours.

3. NEWMAN EXERCISE 2.6

We start by trying to solve for v_2 using the equation

$$v_2^2 - \frac{2GM}{v_1 l_1} v_2 - [(v_1^2 - \frac{2GM}{l_1}] = 0 \tag{6}$$

where $G = 6.67 \cdot 10^{-11}\ m^3\ kg^{-1}\ s^{-2}$, $M = 1.9891 \cdot 10^3 0$ kg.

We know that from this equation, the planet's velocity at its aphelion will be the minimum root. We can write a program to solve this for us. First we sketch out pseudo code

```
define constants

define a
define b
define c

l1 = float(input("Enter
planet's distance at perihelion: "))
v1 = float(input("Enter
planet's velocity at perihelion: "))

solve for the determinant
solve for root 1 using the determinant
solve for root 2 using the determinant
v2 = min([root1,root2])
```

Using the calculated velocity at aphelion, we can calculate the planet's distance at aphelion with

$$l_2 v_2 = l_1 v_1 \tag{7}$$

make sure to define these

From there, we can write a program that calculates the semi-major axis and semi-minor axis using

$$a = \frac{1}{2}(l_1 + l_2) \tag{8}$$

$$b = \sqrt{l_1 l_2} \tag{9}$$

respectively. From there, we can calculate the planet's orbital period as well as its eccentricity

$$T = \frac{2\pi ab}{l_1 v_1} \tag{10}$$

$$e = \frac{l_2 - l_1}{l_2 + l_1} \tag{11}$$

respectively.

Calculating some of the properties of Earth where $l_1 = 1.4710 \cdot 10^1 1$m and $v_1 = 3.0287 \cdot 10^4 ms^-1$

```
l_2: 152203426488.95474 m
v_2: 29271.467816286717 m/s
T: 1.0013930557483963 years
e: 0.017051012575504464
```

Calculating some of the properties of Halley's Comet where $l_1 = 8.7830 \cdot 10^1 0$m and $v_1 = 5.4529 \cdot 10^4 ms^-1$

```
l2: 5282214660876.441 m
v2: 906.6806969191493 m/s
T: 76.08170065465461 years
e: 0.9672889126454061
```

4. SURVEY

The assignment took roughly 6-7 hours. The most difficult part was the LaTeX typesetting and getting everything to fit nicely within the .tex file. The python coding wasn't too difficult.

Overall, nice write-up! Just make sure to define equation variables. Make sure to turn in code!

72.5
80

Computational Physics/Astrophysics, Winter 2023:

Grading Rubrics [1]

Haverford College, Prof. Daniel Grin

For coding assignments, roughly 25 points will be available per problem.

1. Does the program complete without crashing in a reasonable time frame? If yes, up to +3 points.

   2.10 3 2.2 3 2.6 3

2. Does the program use the exact program files given (if given), and produce an answer in the specified format? If yes, +1 points

   2.10 1 2.2 1 2.6 1

3. Does the code follow the problem specifications (i.e numerical method; output requested etc.) Up to +2 points

   2.10 2 2.2 2 2.6 2

4. Is the answer correct? Up to+4points

   2.10 4 2.2 4 2.6 4

5. Is the code readable? Up to+2points

   2.10 2 2.2 2 2.6 2

. 5.1. Are variables named reasonably?

. 5.2. Are the user-functions and imports used?

. 5.3. Are units explained (if necessary)?

. 5.4. Are algorithms found on the internet/book/etc. properly attributed?

[1] Inspired by rubric of D. Narayanan, U. Florida, and C. Cooksey, U. Hawaii

6. Is the code well documented? +3points

2.10 3 2.2 3 2.6 3

. 6.1. Is the code author named?

. 6.2. Are the functions described and ambiguous variables defined?

. 6.3. Is the code functionality (i.e. can I run it easily enough?) documented?

7. LaTeX writeup (up to 10 points)

- define eqn. variables

. Are key figures and numbers from the problem given? (3 points)
. Is a brief explanation of physical context given? (2 points)
. If relevant, are helpful analytic scalings or known solutions given? (1 point)
. Are 3-4 key equations listed (preferably the ones solved in the programming assignment) and algorithms named? (2 points)
. Are collaborators clearly acknowledged? (1 point)
. Are any outside references appropriately cited? (1 point)

2.10 9 2.2 9 2.6 9

Note, even if (1), (2), (3), or (4) are not correct, one can still obtain many points via (5), (6), and (7).

Totals: 2.10 $\frac{22.5}{25}$ 2.2 $\frac{22.5}{25}$ 2.6 $\frac{22.5}{25}$

Code is very well organized!!

-10% of Code questions for late

+5 survey